PROJECT DOCUMENT · VERSION 1.0 · 2026

LOCKINGANG

AI LEARNING STATE ENGINE

PLATFORM Desktop (Electron)

STACK React · Node.js · Python

STATUS In Development

01 EXECUTIVE SUMMARY

lockingang is a local-first desktop application that solves the problem of passive, inefficient study for students and self-learners. Its core — the *Learning State Engine* — continuously models what a student knows, how fast they are forgetting it, and what single action they should take next to maximise long-term retention.

THE CORE PROBLEM

Students spend hours re-reading notes passively without knowing which topics they are actually forgetting. They cannot answer: *What am I genuinely weak at? Am I improving or regressing? What should I focus on right now?* lockingang solves this with a live, per-topic knowledge score that decays in real time, driving personalised quiz selection, intelligent scheduling, and AI-generated guidance.

MATHEMATICAL ANALOGY

Knowledge is modelled as a vector field **V** where each concept is a point. Without active review, entropy causes the magnitude of these vectors (mastery) to shrink toward zero. The engine acts as an *External Impulse Force* that calculates the necessary work (active recall) required to counteract decay and maintain high-energy mastery.

DESIGN PRINCIPLES

1 – CONSTRAINT OVER FREEDOM

The Rule of 3 prevents choice paralysis. The active task queue has a hard limit of 3 items. Limitations are features.

2 – ACTIVE OVER PASSIVE

Every interaction drives active recall. Re-reading is replaced by quizzes, and progress requires demonstrated understanding.

3 – VISUAL OVER TEXTUAL

The Fog of War overlay, forgetting forecasts, and colour-coded nodes make memory decay tangible and intuitive rather than abstract.

4 – COMPASSION OVER SHAME

Bridge nodes and grandparent resets treat failure as structural data points to be solved — not personal flaws. The system adapts to the student.

5 – EXPLAINABLE OVER OPAQUE

Every recommendation comes with reasoning. The student always understands *why* the system is suggesting a particular action.

6 – ADAPTIVE OVER STATIC

The system continuously evolves its model of the student. Long gaps, bursts of activity, and shifting priorities are all handled gracefully.

03 TECHNICAL STACK

LAYER	TECHNOLOGY	PURPOSE
App Shell	`Electron v40`	Cross-platform desktop container
UI Framework	`React 18` + `Tailwind CSS 3`	Component-based UI with custom vector dark theme
Routing	`React Router DOM v7`	HashRouter SPA navigation across 6 tabs
Build Tool	`Vite 7`	Dev server (port 5173) and production bundler
Backend Logic	`Python` (Node.js IPC bridge)	Knowledge graph engine, decay cycles, AI orchestration
Knowledge Graph	Custom DAG (`KnowledgeTree.py`)	Directed graph with BFS traversal, wall detection, bridge generation
AI / LLM	`OpenAI GPT-4o`	Quiz generation, content analysis, summarisation, OCR
Local Embeddings	`Transformers.js` (sidecar HTTP)	768-float semantic vectors, fully local inference
Vector Search (local)	`FAISS IndexFlatIP`	Per-node cosine similarity search, scoped RAG retrieval
Vector Search (cloud)	`Pinecone`	Cloud semantic retrieval for the chatbot
Database	`SQLite + WAL mode`	Persistent node/edge storage, safe concurrent writes
Calendar	`Google Calendar API`	Auto-schedule and display review sessions
Task Management	`Todoist API`	Bidirectional task sync
External Quiz Source	`Kahoot API`	Community quiz content retrieval and adaptation
Auth (Calendar)	`@google-cloud/local-auth`	OAuth 2.0 desktop flow, token.json persistence

04 APPLICATION STRUCTURE & ARCHITECTURE

THREE-LAYER ARCHITECTURE

LAYER	TECHNOLOGY	ENTRY POINT
Main Process	Electron + Node.js	`src/main/main.js`
Renderer (UI)	React SPA (Vite)	`src/renderer/index.jsx`
Backend Engine	Python	`src/backend/node_logic/main.py`

The Electron main process hosts the React renderer via Vite dev server in development or a bundled `dist/index.html` in production. Context isolation is enabled; node integration is disabled. The IPC bridge in `preload.js` exposes safe APIs to the renderer via `window.electron.*`.

IPC HANDLERS (MAIN.JS)

CHANNEL	ACTION
`google-auth`	Trigger OAuth login flow, save token.json
`calendar-get-events`	Fetch upcoming Google Calendar events
`calendar-create-event`	Insert a review block into Google Calendar
`calendar-delete-event`	Remove a review block by event ID

FRONTEND FILE STRUCTURE

PATH	ROLE
`src/renderer/index.jsx`	React root with HashRouter and 6 route definitions
`src/renderer/screens/`	Full-page screen components (one per tab)
`src/renderer/components/`	Shared UI components (Sidebar, Header, FocusTunnel, TaskCard)
`src/renderer/nodeStore.js`	Lightweight reactive state store for the knowledge graph
`src/renderer/styles/global.css`	Global styles, scanline overlay, terminal font helpers

BACKEND FILE STRUCTURE

PATH	ROLE
`src/backend/node_logic/node.py`	Atomic learning unit — mastery, decay, FAISS index, quiz logic

PATH	ROLE
`src/backend/node_logic/KnowledgeTree.py`	Directed graph — traversal, wall detection, scheduling analytics
`src/backend/node_logic/main.py`	Public API (~50 functions) — sole entry point for the IPC layer
`src/backend/kahoot/`	Kahoot search, evaluator, and question adapter pipeline
`src/backend/quiz_generation/`	RAG quiz fallback and diamond synthesis question generators
`src/lib/googleAuth.js`	Google OAuth 2.0 flow with token caching

05 TAB SPECIFICATIONS

TAB	ROUTE	STATUS
Dashboard	`/`	LIVE
Knowledge Tree	`/knowledge-tree`	LIVE
Chatbot	`/chatbot`	LIVE
Calendar & Schedule	`/calendar`	LIVE
Templates	`/templates`	LIVE
Focus Tunnel	`/focus-tunnel`	PARTIAL

DASHBOARD

The first screen a returning student sees. Displays the urgency queue (top nodes most in need of review), a mastery distribution overview, trend analysis (improving / stable / regressing), 7-day forgetting forecast, and integrated chatbot access. Also hosts the Focus Tunnel queue for task management.

KNOWLEDGE TREE

The heart of lockingang. An interactive, zoomable graph where each node represents a concept and each edge a relationship. Nodes are colour-coded by urgency:

BLUE – MASTERED

Live score > 0.7. Node is strong.

YELLOW – FADING

Score 0.3–0.7. Review soon.

RED – CRITICAL

Score < 0.3. Quiz due.

Edge types: `requires` (prerequisite), `is_a_type_of` (taxonomy), `related_to` (association), `bridges_to` (AI-generated transition).

Visual Overlays: Fog of War (nodes < 0.3 are dimmed), Forgetting Forecast (7-day heatmap), Urgency Pulse (animated glow on critical nodes).

FOCUS TUNNEL

A distraction-free task execution environment. The active queue is hard-limited to 3 tasks (Rule of 3). Users add tasks by name and duration. Clicking Start enters Zen Mode — a full-screen countdown timer that counts down from the user-set duration. On completion, the student is prompted to connect the session to a Knowledge Tree node.

CHATBOT

An agentic AI assistant deeply integrated with the student's knowledge graph. Capabilities include RAG-powered Q&A (grounded in the student's own notes via Pinecone), graph manipulation via natural language, calendar and Todoist actions, and multi-day exam study plan generation.

TEMPLATES

A library of pre-built learning roadmaps. Students can browse, import, and clone community-shared trees. Importing clones the full structure (nodes, edges, notes, pre-made quizzes) into the student's own graph. Students can also export and share their own trees as templates.

CALENDAR & SCHEDULE

A unified weekly view combining Google Calendar events with lockingang's auto-scheduled review sessions. Events are colour-coded by node urgency. Review blocks can be dragged to reschedule within a 1-week window. Clicking a review block opens the relevant node's quiz panel.

06 FORGETTING CURVE ENGINE

Every node has a real-time competence score that decays continuously based on how long ago it was last reviewed and the node's individual decay rate.

```
live_score = stored_score × (0.5 ^ (days_elapsed / decay_rate))
```

PARAMETER	DESCRIPTION
`stored_score`	Mastery after the last quiz: 1.0 (correct) or 0.2 (incorrect)
`days_elapsed`	Fractional days since `last_reviewed` timestamp
`decay_rate`	Per-node half-life in days (default 5). Higher = slower decay.
`live_score`	Current effective mastery, clamped to [0.0, 1.0]

A background decay cycle runs hourly. When a node transitions from blue → yellow, a review session is auto-scheduled. When it crosses yellow → red (or is within 24 hours of the scheduled quiz), it turns red and appears at the top of the urgency queue. Completing the quiz resets the decay clock.

07 QUIZ ENGINE

Every quiz interaction updates the student's competence scores and feeds the adaptive logic engine. The quiz flow is:

1. **Kahoot Search** — Query the Kahoot API with the node title as the search term. Retrieve top 5 candidate quizzes.
2. **Kahoot Evaluation** — GPT-4o scores each candidate for relevance (0–1) and difficulty. Accept if relevance ≥ 0.7, adapt/reword questions as needed.
3. **RAG Fallback** — If no Kahoot scores ≥ 0.7, embed the node title, retrieve top 6 chunks from the node's own FAISS index, and generate 5–10 MCQs via GPT-4o grounded strictly in the student's material.
4. **Synthesis Questions (Diamond Problem)** — If the node has 2+ parents, append 1–2 synthesis questions requiring integrated understanding from both parent concepts.
5. **Scoring** — Correct answer → mastery = 1.0. Incorrect → mastery = 0.2. Decay clock resets on every attempt.

08 ADAPTIVE LOGIC ENGINE

The adaptive engine identifies when a student is structurally stuck and intervenes automatically — without blame.

WALL DETECTION

If a student fails a child node's quiz **3+ consecutive times** while the parent node shows mastery (> 0.7), the system flags it as *The Wall*. This indicates a gap in foundational understanding between the parent and child concepts.

BRIDGE NODE GENERATION

Upon Wall detection, the system automatically generates a new *bridge node* between the parent and the difficult child. GPT-4o writes a 300–500 word introductory markdown note focused on the specific gap. The bridge node is inserted into the graph as: `parent → bridges_to → bridge → bridges_to → child`.

GRANDPARENT RESET

If the wall persists (the bridge node already exists and the student still fails), the system escalates by resetting the **grandparent** node's mastery to 0.0, forcing a review of the deeper fundamentals driving the comprehension gap.

09 CONTENT INGESTION PIPELINE

Students can upload raw study materials and the AI automatically structures them into a full knowledge tree.

STEP	ACTION
1. Extraction	Text from PDFs (PyMuPDF), OCR of handwritten images (OpenAI Vision), direct read for Markdown/plaintext
2. Normalisation	GPT-4o converts raw content to clean, structured Markdown
3. Concept ID	GPT-4o identifies key concepts, returns JSON: `{title, description, complexity 1-5}`
4. Tree Generation	GPT-4o infers hierarchical + prerequisite relationships, returns edge list
5. Note Attachment	Relevant sections matched to each concept node
6. Embedding & Indexing	Transformers.js embeds each chunk (768-float), FAISS IndexFlatIP stores vectors per node
7. Student Review	Tree returned in *pending_review* state — student inspects, adjusts, confirms before SQLite commit

10 AUTO-SCHEDULER

Because lockingang tracks per-topic mastery, decay rates, and quiz history, it can schedule study sessions with precision rather than guesswork.

Forgetting Curve Optimiser

- Projects when each node will fall below the critical threshold (0.3)
- Generates optimal review slots before the threshold is crossed
- Auto-inserts review blocks into Google Calendar (colour-coded blue/yellow/red)
- Sessions recalculated daily by the background decay cycle

Exam Preparation Mode

- **Scope Detection:** Identifies all nodes relevant to the exam topic
- **Gap Analysis:** Highlights nodes below target competence (0.7)
- **Plan Generation:** Creates a multi-day plan front-loading weak topics
- **Calendar Injection:** Approved sessions scheduled with direct node links

Calendar Scheduling Rules

NODE STATE	CALENDAR ACTION
YELLOW (score crosses 0.7 → 0.3)	Auto-create review block a few days before projected decay
RED (score < 0.3 or ≤ 24h to quiz)	Update block to today/tomorrow, node turns red
Quiz completed → back to BLUE	Delete the calendar event, reset decay clock
Manual reschedule (drag)	Allowed within 1-week window, updates Calendar event

11 GOOGLE CALENDAR INTEGRATION

Implemented via the Google Calendar API v3 with OAuth 2.0 desktop flow (`@google-cloud/local-auth`). Credentials are stored in `src/backend/calendar_logic/credentials.json` (gitignored). After the first login, a `token.json` is saved alongside it for token reuse.

IPC CHANNEL	DESCRIPTION
`window.electron.googleAuth()`	Trigger OAuth browser login, save token
`window.electron.calendarGetEvents()`	Fetch next 20 upcoming events from primary calendar
`window.electron.calendarCreateEvent(data)`	Insert a review block event
`window.electron.calendarDeleteEvent(id)`	Delete a review block by event ID

IDEAL USER FLOW

1. **First-Time Setup** — Import a template or upload course materials. AI generates the knowledge tree. Connect Google Calendar and Todoist.
2. **Daily Study Session** — Check Dashboard briefing → move top urgency items to Focus Tunnel → enter Zen Mode → complete session. Stray thoughts captured via Brain Dump (Cmd+J). Decay clock recalculates overnight.
3. **Exam Preparation** — Tell the Chatbot your exam date. It proposes an N-day study plan and injects sessions into Google Calendar for confirmation.
4. **Ongoing Adaptation** — Wall Detection automatically inserts bridge nodes when a student repeatedly fails a concept. Grandparent resets escalate to deeper fundamentals when needed.

IMPLEMENTATION STATUS

COMPONENT	STATUS	NOTES
Dashboard Screen	DONE	Dynamic task input, Focus Tunnel, Zen Mode timer
Knowledge Tree Screen	DONE	Graph rendering with dummy data wired
Chatbot Screen	DONE	RAG chatbot UI complete
Calendar Screen	DONE	Weekly grid, Google Calendar fetch, week navigation
Templates Screen	DONE	Browse and import community roadmaps
Google Calendar Auth	DONE	OAuth flow, token.json persistence
Kahoot Pipeline	DONE	search.py, evaluator.py, adapter.py implemented
RAG Quiz Generation	DONE	rag_quiz.py and synthesis.py implemented
node.py / KnowledgeTree.py	DONE	Full decay model, adaptive logic, FAISS, graph traversal
SQLite Persistence	TODO	startup(), shutdown(), _persist_node() stubs remain
run_decay_cycle()	TODO	Hourly background job + Calendar sync not yet wired
Content Ingestion Pipeline	TODO	import_from_ai_generation() body is a stub
Calendar Auto-Scheduling	PARTIAL	IPC handlers done; decay → calendar trigger pending
Brain Dump (Cmd+J)	PARTIAL	UI exists in Zen Mode; global hotkey not registered